



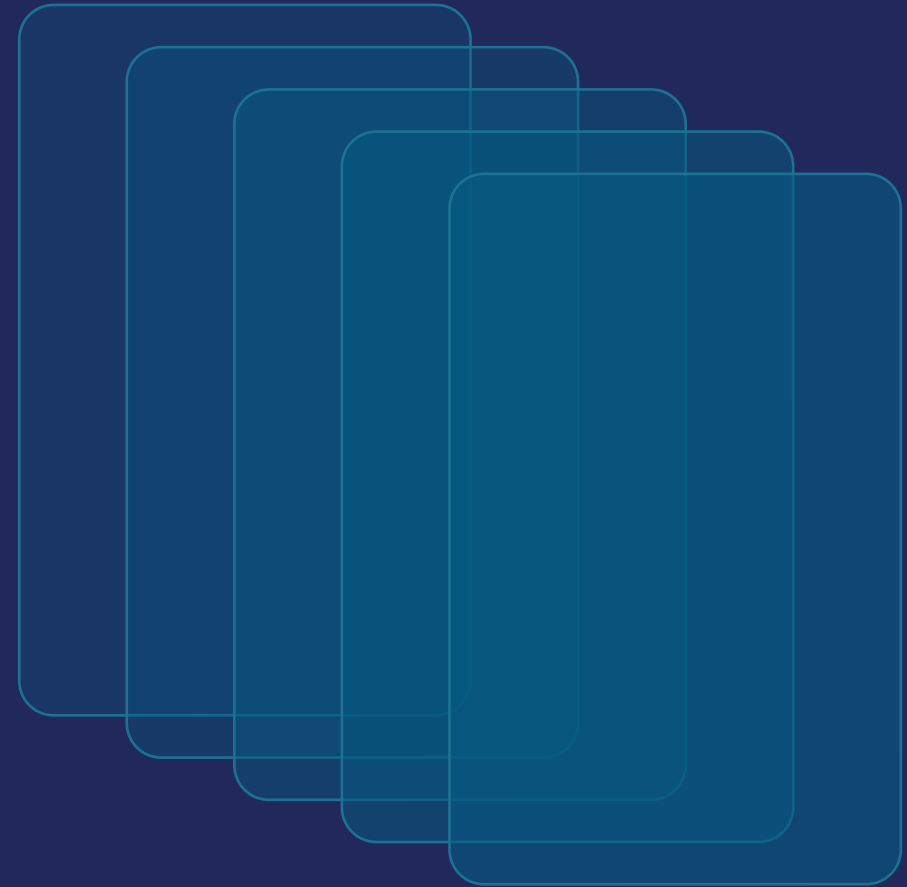
KUBERNETES TRAINING

Module 4:

Namespaces, Labels & Deployments

Partition the cluster, select workloads precisely, and operate Deployments safely.

2026 MODERNIZED EDITION





From Pods + Services to workload management



Module 1

Images package software.
Containers run those images.
Docker gives the runtime workflow.



Module 2

Control plane decides.
Worker nodes execute.
kubectl talks to the API server.



Module 3

Pods are the smallest deployable unit.
Services provide stable discovery.

TODAY

1

Namespaces + ResourceQuota

2

Labels, Selectors + Annotations

3

ReplicaSets + self-healing

4

Deployment hierarchy

5

Dry-run + rolling update

6

Rollout / rollback / scale

7

Broken rollout troubleshooting

What is a Namespace?

A logical partition inside one Kubernetes cluster — useful for separating teams, environments, or application domains.



Names are namespace-scoped for many workload objects; the cluster remains shared.

Default namespaces



default

App

Default home for user workloads when no namespace is specified.



kube-system

Platform

Core Kubernetes system components and controllers.



kube-public

Cluster

Readable-by-all namespace intended for cluster-public information.



kube-node-lease

Node

Node heartbeat / lease objects used for node liveness.

Tip: always know your current namespace before troubleshooting or deleting resources.



Creating a Namespace — namespace.yaml

```
namespace.yaml

apiVersion: v1
kind: Namespace
metadata:
  name: testns
```



Namespace scope

The Namespace object itself is cluster-scoped. Objects such as Pods, Services, ReplicaSets, and Deployments can be created inside it.

Use `-n testns` or set a context namespace when working there.

```
$ Terminal
```

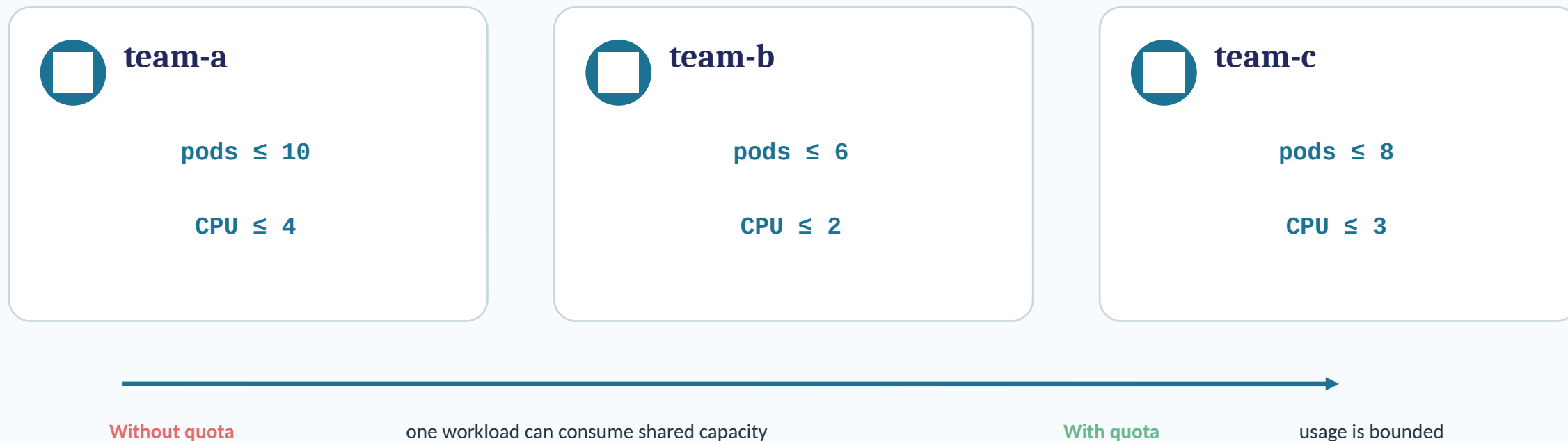
```
kubectl apply -f namespace.yaml
```

OUTPUT

```
namespace/testns created
```

Resource Isolation with ResourceQuota

A namespace can be isolated logically — ResourceQuota adds a consumption guardrail.



ResourceQuota manifest + apply

```
resourcequota.yaml

apiVersion: v1
kind: ResourceQuota
metadata:
  name: testnsquota
  namespace: testns
spec:
  hard:
    pods: "2"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
```

```
$ Terminal
```

```
kubectl apply -f resourcequota.yaml
```

OUTPUT

```
resourcequota/testnsquota created
```



Important

Once a quota tracks CPU or memory, every new Pod must declare requests and limits — or a LimitRange must supply defaults.



Verifying and testing a ResourceQuota

Terminal

```
kubectl get resourcequota -n testns
```

OUTPUT

NAME	REQUEST
testnsquota	Pods: 1/2, requests.cpu: 100m/1, ...

Terminal

```
kubectl describe resourcequota  
testnsquota -n testns
```

OUTPUT

Name:	testnsquota	
Resource	Used	Hard
limits.memory	128Mi	2Gi
Pods	1	2
requests.cpu	100m	1
requests.memory	64Mi	1Gi

Terminal

```
kubectl apply -f highmem-pod.yaml
```

OUTPUT

Error from server (Forbidden): pods "highmem-pod" is forbidden: exceeded quota: testnsquota, requested: requests.memory=2Gi, used: requests.memory=64Mi, limited: requests.memory=1Gi

What are Labels?

Labels are key-value tags attached to Kubernetes objects for organizing and selecting resources.

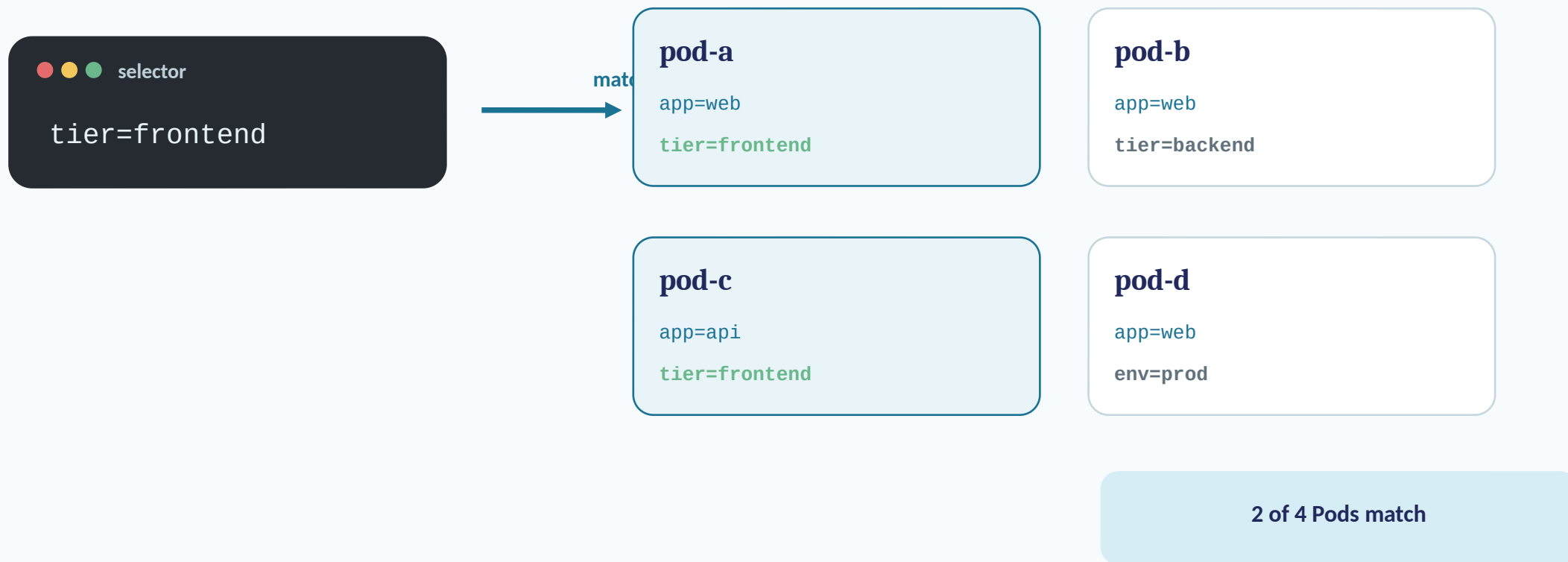


Same label can appear on many objects — labels are not unique identifiers.



What are Selectors?

A selector filters resources by label; controllers and Services use this relationship to target objects.





Working with Labels via kubectl

Terminal

```
kubectl get pods --show-labels
```

OUTPUT

NAME	READY	STATUS	LABELS
web-7f	1/1	Running	app=web, tier=frontend

Terminal

```
kubectl get pods -l tier=frontend
```

OUTPUT

NAME	READY	STATUS	AGE
web-7f	1/1	Running	2m
web-9k	1/1	Running	2m

Terminal

```
kubectl label pod web-7f env=production  
kubectl label pod web-7f env-
```

OUTPUT

```
pod/web-7f labeled  
pod/web-7f unlabeled
```



Annotations are metadata for tools and humans



Labels

Selection + grouping

- app=web
- tier=backend
- environment=prod

Controllers / Services can select by label.



Annotations

Rich metadata; not used for selection

- build/version info
- tool configuration
- Git commit / timestamps

Use them for context and integration.

Rule of thumb: “Should Kubernetes select this object?” → Label. “Should a tool read this metadata?” → Annotation.



Working with Annotations

Terminal

```
kubectl annotate pod annotation-demo \
  description="demo pod for module 4"
kubectl get pod annotation-demo \
  -o jsonpath='{.metadata.annotations}'
```

OUTPUT

```
{"build-version":"1.2.3",
  "description":"demo pod for module 4",
  "owner":"dev-team"}
```

Terminal

```
kubectl annotate pod annotation-demo description-
```

OUTPUT

```
pod/annotation-demo annotated
```

Terminal

```
kubectl get pod annotation-demo -o yaml
```

OUTPUT

```
annotations:
  build-version: "1.2.3"
  owner: dev-team
```

What is a ReplicaSet?

A ReplicaSet continuously works toward a desired number of matching Pods.



Selector → finds matching Pods • Template → defines replacement Pods • replicas → desired count



ReplicaSet YAML manifest

```
nginx-rs-replicaset.yaml

apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replica-set
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.28-alpine
          ports:
            - containerPort: 80
```

```
$ Terminal
```

```
kubectl apply -f nginx-rs-
replicaset.yaml
```

OUTPUT

```
replicaset.apps/nginx-replica-set
created
```



Selector ↔ template

The selector must match the labels in the Pod template.

That relationship is how the ReplicaSet knows which Pods it owns.

Verify and test self-healing

 \$ Terminal

```
kubectl get rs
```

OUTPUT

NAME	DESIRED	READY
nginx-replica-set	3	3

 \$ Terminal

```
kubectl get pods
```

OUTPUT

nginx-replica-set-8dttg	Running
nginx-replica-set-h7jmt	Running
nginx-replica-set-v6lsx	Running

 \$ Terminal

```
POD_NAME=$(kubectl get pods -l app=nginx -o  
jsonpath='{.items[0].metadata.name}')  
kubectl delete pod $POD_NAME
```

OUTPUT

```
pod "nginx-replica-set-8dttg" deleted  
from default namespace
```

 \$ Terminal

```
kubectl get pods --watch
```

OUTPUT

...-8dttg	1/1	Terminating
...-g5hg7	0/1	ContainerCreating → Running



Cleaning up a ReplicaSet

Terminal

```
kubectl delete -f nginx-rs-replicaset.yaml
```

OUTPUT

```
replicaset.apps/nginx-replica-set deleted
```



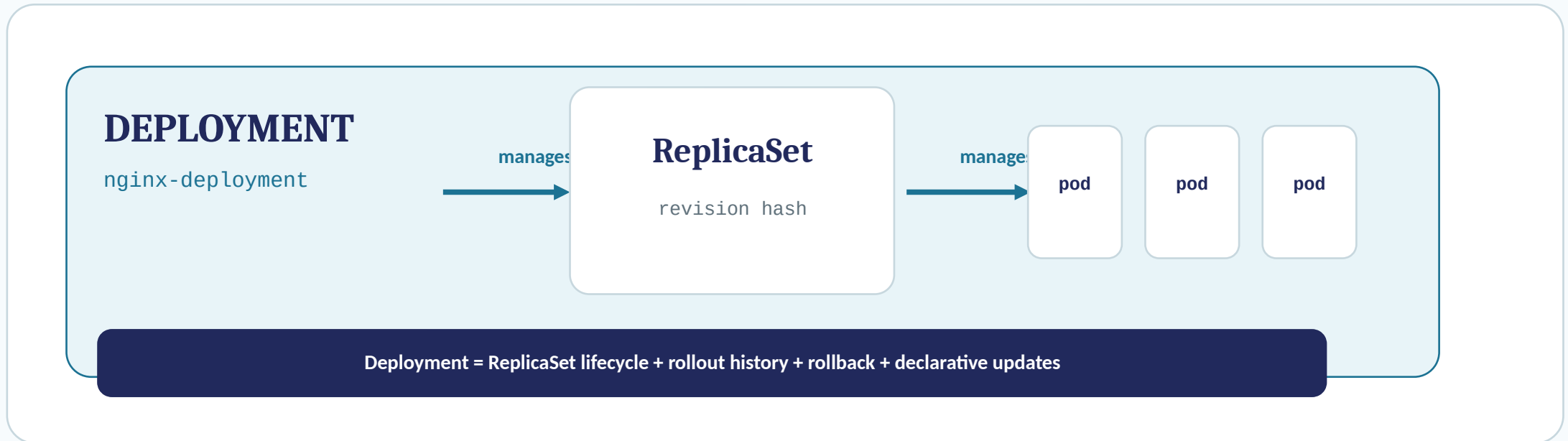
Why use -f?

Deleting from the same manifest you applied is repeatable and keeps the cleanup target explicit. The ReplicaSet owns its Pods, so the managed Pods are removed with it.



Why Deployments, not just ReplicaSets?

ReplicaSets keep a replica count healthy. Deployments add controlled change over time.



What is a Deployment?

A Deployment is the declarative controller you use to manage an application over time — not just to create Pods once.



Declarative updates

Desired state lives in YAML; Kubernetes reconciles toward it.



Rollout history

Each Pod-template change creates a new ReplicaSet revision.



Self-healing + scale

The managed ReplicaSet maintains the requested replica count.



Rollback

Return to a previous revision when an update goes wrong.

Deployment → ReplicaSet → Pods

You declare the desired state; controllers perform the ongoing work.



Your first Deployment — full YAML

```
deployment.yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.28-alpine
          ports:
            - containerPort: 80
  strategy:
    type: RollingUpdate
```



Manifest mental model

selector.matchLabels must match
template.metadata.labels.

replicas = desired Pod count.



Pinned image

Use a concrete tag such as nginx:1.28-alpine —
never :latest — so every rollout is reproducible.

5 replicas • RollingUpdate



Applying and inspecting a Deployment

\$ Terminal

```
kubectl apply -f deployment.yaml
```

OUTPUT

```
deployment.apps/  
nginx-deployment created
```

\$ Terminal

```
kubectl get deploy
```

\$ Terminal

```
kubectl describe deployment  
nginx-deployment
```

OUTPUT

```
Name:                nginx-deployment  
Replicas:            5 desired | 5 updated | 5 total | 5 available  
StrategyType:        RollingUpdate  
Pod Template:        app=nginx
```

OUTPUT

\$ Terminal

```
kubectl get pods
```

OUTPUT

NAME	READY	STATUS	AGE	
nginx-deployment-7546988c87-5cz88	1/1	Running	40s	(×5)



Dry-run before applying — make safety a habit

Dry-run shows the result without saving it: client = local render, server = full API validation.

 \$ Terminal

```
kubectl apply --dry-run=server \  
-f deployment.yaml
```

OUTPUT

```
deployment.apps/nginx-deployment  
configured (server dry run)
```



Then run `kubectl diff -f deployment.yaml` to see exactly which fields will change before you apply.



Rolling Update Strategy — replace safely

Change the Pod template (for example, image tag) and the Deployment creates a new ReplicaSet.



Default strategy: RollingUpdate • gradually scale the new RS up while the old RS scales down.



Watching a rollout live

Terminal

```
kubectl get pods --watch
```

OUTPUT

nginx-7c8d-aaa	1/1	Terminating
nginx-6f4a-ddd	0/1	Pending
nginx-6f4a-ddd	1/1	Running

Terminal

```
kubectl rollout status deployment/nginx-deployment
```

OUTPUT

```
Waiting for deployment "nginx-deployment" rollout to finish:  
 2 out of 5 new replicas have been updated...  
deployment "nginx-deployment" successfully rolled out
```



Operational habit

Use `--watch` for the moving picture and `rollout status` for a clear success/failure signal. Stop the watch with Ctrl+C.



Rollout history & rollback

Terminal

```
kubectl rollout history deployment/nginx-deployment
```

OUTPUT

REVISION	CHANGE-CAUSE
1	Initial deploy nginx 1.28-alpine
2	Upgrade nginx to 1.29-alpine

Terminal

```
kubectl rollout undo deployment/nginx-deployment
```

OUTPUT

```
deployment.apps/nginx-deployment rolled back
```

Terminal

```
kubectl rollout undo deployment/nginx-deployment --  
to-revision=1
```

OUTPUT

```
deployment.apps/nginx-deployment rolled back  
# history rennumbers: revision 1 becomes 3
```

Tip: set the `kubernetes.io/change-cause` annotation in the YAML so the `CHANGE-CAUSE` column explains every revision.



Scaling a Deployment — up and down

Terminal

```
kubectl scale deploy/nginx-deployment --replicas=3
```

OUTPUT

```
deployment.apps/nginx-deployment scaled
```

Terminal

```
kubectl get deploy
```

OUTPUT

NAME	READY	UP-TO-DATE	AVAILABLE
nginx-deployment	3/3	3	3

Terminal

```
kubectl scale deploy/nginx-deployment --replicas=5
```

OUTPUT

```
deployment.apps/nginx-deployment scaled
```

kubectl scale changes only the live object — the next kubectl apply of the YAML resets replicas to the file's value.



Troubleshooting a broken Deployment

1	Detect	<pre>kubectl get pods</pre>	<pre>...-dcbd9d64f-5drzn 0/1 ImagePullBackOff</pre>
2	Inspect	<pre>kubectl describe pod <pod-name></pre>	<pre>Failed to pull image "nginx:1.99-doesnotexist"</pre>
3	Check logs	<pre>kubectl logs <pod-name></pre>	<pre>waiting to start: trying and failing to pull image</pre>
4	Recover	<pre>kubectl rollout undo deploy/nginx-deployment</pre>	<pre>deployment.apps/nginx-deployment rolled back</pre>
5	Fix	<pre>kubectl apply -f deployment.yaml</pre>	<pre>fixed tag in Git → new revision rolls out</pre>

Describe → Events explains image-pull failures; kubectl logs has nothing yet because the container never started.



The ImagePullBackOff teaching moment

Terminal

```
kubectl get pods
```

OUTPUT

NAME	READY	STATUS	AGE
nginx-deployment-dcbd9d64f-5drzn	0/1	ImagePullBackOff	38s

Terminal

```
kubectl describe pod \
  nginx-deployment-dcbd9d64f-5drzn
```

OUTPUT

Events:

Type	Reason	Message
Normal	Pulling	Pulling image "nginx:1.99-doesnotexist"
Warning	Failed	Failed to pull image "nginx:1.99-doesnotexist": ... nginx:1.99-doesnotexist: not found
Warning	Failed	Error: ErrImagePull
Normal	BackOff	Back-off pulling image "nginx:1.99-doesnotexist"
Warning	Failed	Error: ImagePullBackOff

Terminal

```
kubectl logs <pod-name>
```

OUTPUT

```
Error from server (BadRequest): container "nginx" in pod "...-5drzn"
is waiting to start: trying and failing to pull image
```



Cleaning up — delete the Deployment

Terminal

```
kubectl delete deployment nginx-deployment
```

OUTPUT

```
deployment.apps "nginx-deployment" deleted  
from default namespace
```



Cleanup behavior

A Deployment owns its ReplicaSets; each ReplicaSet owns its Pods. Deleting the Deployment removes the whole tree — Kubernetes garbage-collects the children through their ownerReferences.

Try It Yourself — Namespace → Deploy → Roll back

1

Namespace

```
kubectl create namespace training
kubectl config set-context --current --namespace=training
```

```
namespace/training created
```

3

Scale

```
kubectl scale deployment nginx-deployment --replicas=3
```

```
deployment.apps/nginx-deployment scaled
```

5

Verify / undo

```
kubectl rollout status deploy/nginx-deployment
kubectl rollout undo deploy/nginx-deployment
```

```
successfully rolled out / rolled back
```

2

Deploy

```
kubectl apply -f deployment.yaml
```

```
deployment.apps/nginx-deployment created
```

4

Roll

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.29-alpine
```

```
.../nginx-deployment image updated
```

Lab objective: watch one Deployment create a ReplicaSet, scale it, roll to a new revision, then roll back safely.



Lab: when the quota says no

Terminal

```
kubectl run nores -n testns \  
  --image=nginx:1.28-alpine
```

OUTPUT

```
Error from server (Forbidden): pods "nores" is forbidden:  
failed quota: testnsquota: must specify limits.cpu for: nores;  
limits.memory for: nores; requests.cpu for: nores; ...
```

Terminal

```
kubectl apply -f limitrange.yaml  
kubectl run web1 -n testns \  
  --image=nginx:1.28-alpine
```

OUTPUT

```
limitrange/defaults created  
pod/web1 created  
# the LimitRange filled in the missing values:  
{"limits":{"cpu":"200m","memory":"128Mi"},  
 "requests":{"cpu":"100m","memory":"64Mi"}}
```

Terminal

```
kubectl create deployment qdemo -n testns \  
  --image=nginx:1.28-alpine --replicas=2
```

OUTPUT

```
deployment.apps/qdemo created          # READY 0/2  
ReplicaFailure FailedCreate: exceeded quota: testnsquota,  
  requested: pods=1, used: pods=2, limited: pods=2
```



Lab: label selectors that scale

Terminal

```
kubectl get pods -l app=web -L tier,env
```

OUTPUT

NAME	TIER	ENV
pod-a	frontend	prod
pod-b	frontend	staging
pod-d	backend	

Terminal

```
kubectl get pods -o name \
-l 'app=web,tier!=backend'
```

OUTPUT

```
pod/pod-a
pod/pod-b
```

Terminal

```
kubectl get pods -o name -l 'env in (prod)'
kubectl get pods -o name -l '!env'
```

OUTPUT

```
pod/pod-a    pod/pod-c
pod/pod-d    # has no env label
```



Lab: ReplicaSets own their Pods

Terminal

```
kubectl get pods -l app=nginx-rs -o \
  custom-columns=NAME:.metadata.name,\
  OWNER:.metadata.ownerReferences[0].name
```

OUTPUT

NAME	OWNER
nginx-rs-8dttg	nginx-rs
nginx-rs-h7jmt	nginx-rs
nginx-rs-v6lsx	nginx-rs

Terminal

```
kubectl run stray --image=nginx:1.28-alpine \
  --labels=app=nginx-rs
```

OUTPUT

```
pod/stray created
# 2 s later it is gone: the ReplicaSet saw 4 Pods,
# wanted 3, and deleted one (SuccessfulDelete)
```

Terminal

```
kubectl delete rs nginx-rs --cascade=orphan
kubectl apply -f nginx-rs.yaml
```

OUTPUT

```
# Pods kept running with OWNER <none> —
# the new ReplicaSet adopted all 3, no restarts
```



Lab: a production-ready Deployment

deployment.yaml (compact)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  annotations:
    kubernetes.io/change-cause: "Initial deploy nginx 1.28-alpine"
spec:
  replicas: 4
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 60
  selector:
    matchLabels: {app: nginx}
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  template:
    metadata:
      labels: {app: nginx}
    spec:
      containers:
        - name: nginx
          image: nginx:1.28-alpine
          resources:
            requests: {cpu: 50m, memory: 32Mi}
            limits: {cpu: 200m, memory: 128Mi}
          readinessProbe:
            httpGet: {path: /, port: 80}
```



Why each field

change-cause → readable history
revisionHistoryLimit → old RS kept
progressDeadline → stuck = failed
maxUnavailable 0 → never below 4



Readiness gates traffic

New Pods get traffic only after the probe passes
— so maxUnavailable: 0 means no capacity dip.

4 replicas • surge 1 • unavailable 0



Lab: diff first, then roll out

Terminal

```
kubectl diff -f deployment-v2.yaml
```

OUTPUT

```
- ...change-cause: Initial deploy nginx 1.28-alpine
+ ...change-cause: Upgrade nginx to 1.29-alpine
-   - image: nginx:1.28-alpine
+   - image: nginx:1.29-alpine
```

Terminal

```
kubectl apply -f deployment-v2.yaml
kubectl get rs -l app=nginx
```

OUTPUT

NAME	DESIRED	READY
nginx-deployment-5c4d6d6d66	4	4
nginx-deployment-7546988c87	0	0

Terminal

```
kubectl rollout history deploy/nginx-deployment
```

OUTPUT

REVISION	CHANGE-CAUSE
1	Initial deploy nginx 1.28-alpine
2	Upgrade nginx to 1.29-alpine

The old ReplicaSet stays at 0 replicas — that is what makes an instant rollback possible (revisionHistoryLimit keeps 5).



Lab: a broken rollout that hurt nobody

Terminal

```
kubectl set image deploy/nginx-deployment \
  nginx=nginx:1.99-doesnotexist
kubectl rollout status deploy/nginx-deployment
```

OUTPUT

```
deployment.apps/nginx-deployment image updated
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 4...
error: deployment "nginx-deployment" exceeded its progress deadline
```

Terminal

```
kubectl get pods -l app=nginx
```

OUTPUT

NAME	READY	STATUS
nginx-deployment-5c4d6d6d66-5cz88	1/1	Running (x4)
nginx-deployment-dcbd9d64f-5drzn	0/1	ErrImagePull

maxUnavailable: 0 → all 4 old Pods still serve traffic

Terminal

```
kubectl describe deploy nginx-deployment
```

OUTPUT

Progressing	False	ProgressDeadlineExceeded
Available	True	MinimumReplicasAvailable



Lab: roll back — then fix Git

Terminal

```
kubectl rollout undo deploy/nginx-deployment
```

OUTPUT

```
Warning: resource deployments/nginx-deployment was  
previously managed with 'kubectl apply'. Rolling back  
will not update ... last-applied-configuration ...  
deployment.apps/nginx-deployment rolled back
```

Terminal

```
kubectl rollout history deploy/nginx-deployment
```

OUTPUT

```
1  Initial deploy nginx 1.28-alpine  
3  Bad release: nginx:1.99-doesnotexist  
4  Upgrade nginx to 1.29-alpine
```

Terminal

```
kubectl rollout undo deploy/nginx-deployment \  
--to-revision=1
```

OUTPUT

```
deployment.apps/nginx-deployment rolled back  
# revision 1 comes back as revision 5
```

Undo is the emergency brake. Then fix the YAML in Git and apply it — or the next apply brings the bad tag back.



Deployments in 2026 — what's new



In-Place Pod Resize — GA in v1.35

Change CPU and memory of a running Pod with `kubectl patch --subresource resize` — no restart needed.

Deployments: resizing one Pod does not change the Pod template. The next rollout recreates Pods with the template's values, so update the YAML too.



Rollout visibility — v1.37

`status.terminatingReplicas` (beta, on by default) counts old Pods that are still shutting down — a rollout can say “successfully rolled out” while they finish.

StatefulSets also gained an alpha Recreate update strategy (feature gate `StatefulSetRecreateStrategy`). It is off by default and does not apply to Deployments.

2026 takeaway: resize in place for quick fixes, but keep the YAML as the source of truth for every rollout.



Common pitfalls — catch these before production



Selector mismatch

Deployment selector does not match template labels.

Rejected / cannot manage the intended Pods.



Unpinned image

Using :latest hides exactly what changed.

Rollouts become harder to reproduce.



Manual scale drift

kubectl scale changes live state only.

Manifest reapply can reset the replica count.



Blind rollback

Undo without checking history / revision.

You may restore the wrong version.



Quota surprise

Deployment created, but no Pods appear.

ReplicaSet events: exceeded quota.



Bad image tag

ImagePullBackOff during rollout.

Inspect describe → Events; fix or undo.



Module recap — the workload control stack



Namespace

Partition the cluster logically.



ResourceQuota

Bound namespace resource consumption.



Labels / Selectors

Tag objects and target the right ones.



Annotations

Attach tool / human metadata.



ReplicaSet

Maintain N matching Pods.



Deployment

Manage ReplicaSets + change over time.



Rollout

Watch progressive replacement.



Rollback

Return to a known revision.



Scale

Change desired replica count.

Mental model: Namespace scopes → labels select → ReplicaSet maintains → Deployment changes safely.



KNOWLEDGE CHECK

9 questions

Before you move on

Real namespace, label and Deployment problems, with the terminal output you'd see. Answer key on the last quiz slide.

QUOTA

limits

LABELS

select

ROLLOUT

recovers



Question 1 of 9

The Deployment was created, but it stays at READY 0/2. What do you check first?

🔴 🟡 🟢 \$ Scenario

```
k get deploy api -n team-a
```

OUTPUT

NAME	READY	UP-TO-DATE	AVAILABLE
api	0/2	0	0

🔴 🟡 🟢 Think about

Who actually creates the Pods?

CHOOSE ONE

- A. The container logs of the api Pods
- B. The node's free disk space
- C. Deployment conditions and ReplicaSet events — e.g. exceeded quota
- D. Delete the Deployment and apply it again



Question 2 of 9

A team's namespace just got a ResourceQuota. Now every new Pod is rejected. What is the clean fix?

Scenario

```
k run web -n team-a \
  --image=nginx:1.28-alpine
```

OUTPUT

```
forbidden: failed quota: team-quota:
must specify limits.cpu for: web;
requests.memory for: web; ...
```

Think about

A quota needs numbers to count

CHOOSE ONE

- A. Delete the ResourceQuota
- B. Run the Pods in the default namespace instead
- C. Raise the quota to a very large number
- D. Add requests/limits to the Pod spec, or a LimitRange with defaults



Question 3 of 9

You start a debug Pod with label app=checkout. Two seconds later it's gone, with no error. Why?

 \$ Scenario

```
k run debug --image=busybox:1.37 \  
  --labels=app=checkout -- sleep 3600
```

OUTPUT

```
pod/debug created  
$ k get pod debug  
Error from server (NotFound): pods "debug" not found
```

 Think about

Selectors count every matching Pod

CHOOSE ONE

- A. The scheduler found no free node
- B. BusyBox exits immediately
- C. A ReplicaSet selecting app=checkout counted it as extra and deleted it
- D. kubectl run does not allow labels



Question 4 of 9

A rollout failed like this, yet users saw no errors. Why — and what's next?

🔴 🟡 🟢 \$ Scenario

```
k rollout status deploy/web
```

OUTPUT

```
error: deployment "web" exceeded its  
progress deadline  
READY 4/4    UP-TO-DATE 1
```

🔴 🟡 🟢 Think about

Old ReplicaSet vs new ReplicaSet

CHOOSE ONE

- A. maxUnavailable: 0 kept the old Pods serving; read the new Pod's events, then fix or undo
- B. Kubernetes already rolled back automatically
- C. The deadline deleted the new ReplicaSet
- D. Nothing — the rollout will finish by itself



Question 5 of 9

You fixed an outage with rollout undo. Next morning CI runs `kubectl apply` and the bad image is back. Why?

Scenario

```
k rollout undo deploy/web
```

OUTPUT

```
Warning: ... Rolling back will not update  
the last-applied-configuration annotation  
deployment.apps/web rolled back
```

Think about

Where does the truth live?

CHOOSE ONE

- A. CI used an old kubectl version
- B. undo changes only the live object — the YAML in Git still has the bad tag
- C. The revision history was full
- D. rollout undo only lasts 24 hours



Question 6 of 9

During a sale you ran `kubectl scale --replicas=10`. An hour later there are 4 Pods again. Most likely cause?

 \$ Scenario

```
k scale deploy/shop --replicas=10
```

OUTPUT

```
deployment.apps/shop scaled  
# one hour later: READY 4/4
```

 Think about

Imperative vs declarative

CHOOSE ONE

- A. A pipeline re-applied the YAML, which says replicas: 4
- B. Pods expire after one hour
- C. ReplicaSets ignore scale commands
- D. The ResourceQuota deleted the extra Pods



Question 7 of 9

You want to record the Git commit SHA and the build URL on every Deployment. Where do they belong?

🔴 🟡 🟢 \$ Scenario

```
k get deploy web -o yaml
```

🔴 🟡 🟢 Think about

Select vs describe

OUTPUT

```
metadata:  
  labels: {app: web}  
  ????:  commit + build URL
```

CHOOSE ONE

- A. Labels, so you can select by them
- B. Inside the image name
- C. Only in the Pod's environment variables
- D. Annotations — metadata for tools and people, not for selection



Question 8 of 9

Your team says the app is running, but `kubectl get pods` shows nothing. What's the most likely reason?

Scenario

```
k get pods
```

OUTPUT

```
No resources found in default namespace.
```

Think about

Read the end of the message

CHOOSE ONE

- A. You're in the wrong namespace — add `-n`, or set your context's namespace
- B. The cluster is down
- C. Pods are hidden unless you are cluster-admin
- D. Deployments don't create Pods



Question 9 of 9

replicas: 4, maxSurge: 1, maxUnavailable: 0. During a rollout, what are the most Pods and the fewest ready Pods?

Scenario

```
replicas: 4
maxSurge: 1  maxUnavailable: 0
```

OUTPUT

NAME	READY	STATUS
web-old x4	1/1	Running
web-new x1	0/1	ContainerCreating

Think about

Surge adds, unavailable subtracts

CHOOSE ONE

- A. At most 4 Pods, at least 3 ready
- B. At most 5 Pods, at least 4 ready
- C. At most 8 Pods, at least 0 ready
- D. At most 5 Pods, at least 3 ready



Answer key

Score yourself

9 questions
1 point each

ANSWERS

Q1 C quota blocks the ReplicaSet → slide 31 • Step 4
Q2 D requests/limits or LimitRange → slide 31 • Step 4
Q3 C the RS deleted the extra Pod → slide 33 • Step 8
Q4 A old Pods kept serving → slide 36 • Step 11
Q5 B undo is not a Git change → slide 37 • Step 12
Q6 A apply reset the replicas → slide 26 • Step 13
Q7 D annotations for tool data → slide 12 • Step 6
Q8 A wrong namespace → slide 4 • Step 2
Q9 B max 5 Pods, 4 always ready → slide 23 • Step 9

8-9

Module 4B ready

6-7

re-read misses

≤ 5

redo Steps 3-13



Next: Module 4B

Service Types Deep-Dive

You can now partition workloads, select them precisely, maintain replicas, and operate Deployments through change and failure.

NAMESPACE

scopes

LABELS

select

DEPLOYMENT

changes